



南開大學
Nankai University

数据结构实验报告四

链表和栈

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网络安全学院

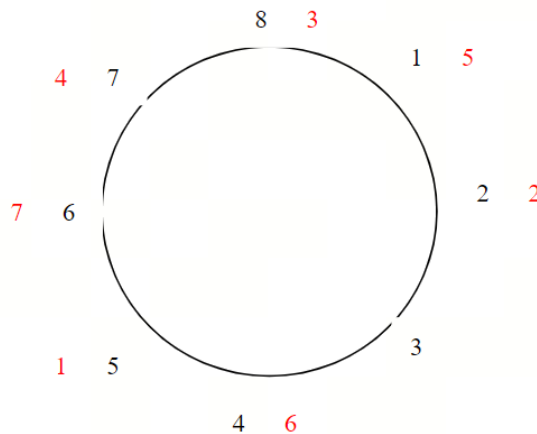
目录

1 题目一	2
1.1 题目表述	2
1.2 代码 1 的测试用例	2
1.3 思路	2
1.3.1 使用数组模拟实现 Josephus	2
1.3.2 使用循环双链表实现 Josephus	3
1.4 代码	3
1.5 测试用例截图	6
1.6 复杂度分析	7
2 题目二	7
2.1 题目表述	7
2.2 代码 2 的测试用例	7
2.3 思路	8
2.4 测试用例截图	9
2.5 代码	10
2.6 复杂度分析	13
2.6.1 空间复杂度	13
2.6.2 时间复杂度	13

1 题目一

1.1 题目表述

Josephus 问题， n 个人围坐成一圈，按顺序编号为 $1 - n$ ，确定一个整数 m ，从 1 号开始数数，每数到第 m 个人出列，剩下的人从下一个人重新开始数，直至只剩一个人为止。对 $n=8, m=5$ ，过程和结果如下图所示，黑色数字为编号，红色数字为出列顺序，最后剩下的是 3 号。



编写程序，对任意输入的 n 和 m ，求出最后剩下的人的编号。要求利用线性表保存这 n 个人，分别用公式化和链表两种描述方法实现。输入：input.txt，两个整数 n ($3 - 100$)， m ($1 - m$)

输出：若输入合法，按出列顺序输出人的编号，否则输出 “WRONG”。编号之间用一个空格间隔，最后一个编号后不能有空格。两种实现方法各输出一次，用回车间隔，最后输出一个回车。

1.2 代码 1 的测试用例

1. 输入： $n=3, m=1$ 输出：1 2 3
2. 输入： $n=10, m=1000$ 输出：10 1 9 7 4 3 2 5 8 6
3. 输入： $n=2, m=-1$ 输出：WRONG
4. 输入： $n=100, m=97$ 输出太长，最后一个是 55
5. 输入： $n=5, m=5$ 输出：5 1 3 4 2

1.3 思路

1.3.1 使用数组模拟实现 Josephus

使用一个动态数组 `arr[]` 存储当前还在的人的索引，每次计算下一个被淘汰者的索引并且通过数组前移将其移除，最后输出淘汰顺序

```
1 current = (current + m - 1) % count;
```

1.3.2 使用循环双链表实现 Josephus

1. 构造链表

创建一个长度为 n 的循环双向链表，节点值为 1 到 n , $head \rightarrow prev = tail$, $tail \rightarrow next = head$ 链表首尾相连。

2. 执行 Josephus

从 $head$ 开始，顺时针移动 $m-1$ 步，找到要删除的节点 $current$ ，输出其值。之后调整前后指针，删除该节点，如果删除的是头节点更新 $head$ 。然后 $size--$ ，继续直到只剩一人。

1.4 代码

```
1 #include<iostream>
2 #include<fstream>
3 #include<string>
4 using namespace std;
5
6 class Node {
7 public:
8     int data;
9     Node* prev;
10    Node* next;
11    Node() :data(0), prev(NULL), next(NULL) {};
12    Node(int n) :data(n), prev(NULL), next(NULL) {};
13 };
14
15 class circulardoublylinkedlist {
16 public:
17     Node* head;
18     int size;
19
20 public:
21     circulardoublylinkedlist(int n);
22     ~circulardoublylinkedlist();
23     void Josephus(int m);
24 };
25
26 circulardoublylinkedlist::circulardoublylinkedlist(int n) {
27     size = n;
28     head = new Node(1);
29     Node* current = head;
30     for (int i = 2; i <= n; i++) {
31         Node* newNode = new Node(i);
```

```
32     current->next = newNode;
33     newNode->prev = current;
34     current = newNode;
35 }
36 current->next = head;
37 head->prev = current;
38 }
39
40 circular doubly linked list::~circular doubly linked list() {
41     if (head == NULL) return;
42     Node* current = head;
43     Node* toDelete;
44     do {
45         toDelete = current;
46         current = current->next;
47         delete toDelete;
48     } while (current != head);
49 }
50
51 void circular doubly linked list::Josephus(int m) {
52     if (head == NULL || size == 0) return;
53     Node* current = head;
54     while (size > 1) {
55         for (int i = 1; i < m; i++) {
56             current = current->next;
57         }
58         cout << current->data;
59         if (size > 2) cout << " ";
60         Node* toDelete = current;
61         current->prev->next = current->next;
62         current->next->prev = current->prev;
63         current = current->next;
64         if (toDelete == head) {
65             head = current;
66         }
67
68         delete toDelete;
69         size--;
70     }
71     cout << " " << head->data << endl;
72 }
73
```

```
74 // 数组实现 (公式化描述)
75 void arrayJosephus(int n, int m) {
76     int* arr = new int[n];
77     for (int i = 0; i < n; i++) {
78         arr[i] = i + 1;
79     }
80
81     int current = 0;
82     int count = n;
83
84     while (count > 1) {
85         current = (current + m - 1) % count;
86         cout << arr[current];
87         if (count > 2) cout << " ";
88         for (int i = current; i < count - 1; i++) {
89             arr[i] = arr[i + 1];
90         }
91         count--;
92     }
93     cout << " "<<arr[0] << endl;
94     delete[] arr;
95 }
96
97 int main() {
98     int n, m;
99     ifstream infile("input.txt");
100     infile >> n >> m;
101     infile.close();
102     cout << "-----"<<endl;
103     if (n < 3 || n > 100 || m < 1 ) {
104         cout << "WRONG" << endl;
105         cout << endl << "-----" << endl;
106         system("pause");
107         return 0;
108     }
109     arrayJosephus(n, m);
110     circular doubly linked list list(n);
111     list.Josephus(m);
112     cout << endl << "-----" << endl;
113     system("pause");
114     return 0;
115 }
```

1.5 测试用例截图



图 1.1: 题目一测试用例输出结果 1



图 1.2: 题目一测试用例输出结果 2



图 1.3: 题目一测试用例输出结果 3

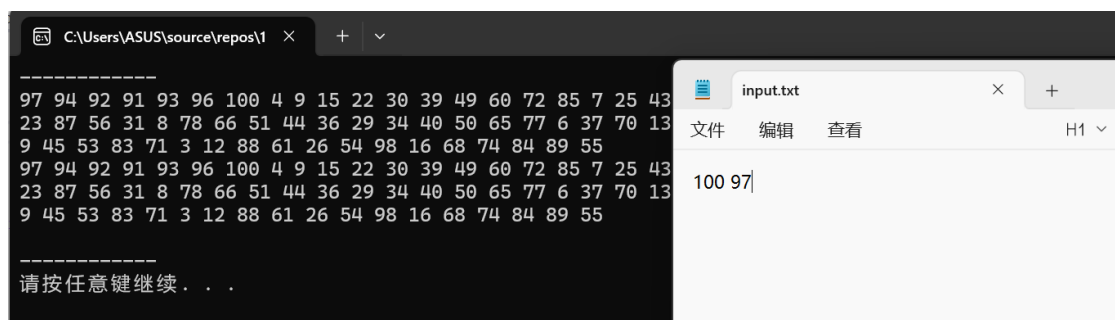


图 1.4: 题目一测试用例输出结果 4

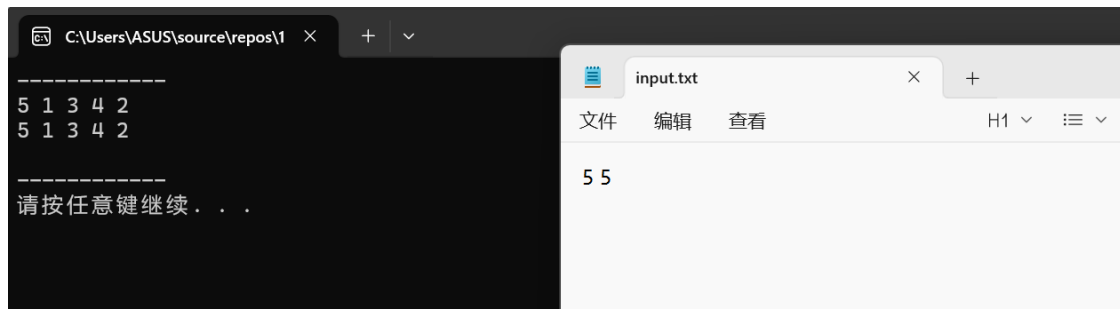


图 1.5: 题目一测试用例输出结果 5

1.6 复杂度分析

方法	时间复杂度	空间复杂度	说明
数组实现	$O(n^2)$	$O(n)$	每次删除需 $O(n)$ 移动元素，共删除 $n-1$ 次
链表实现	$O(n \times m)$	$O(n)$	每次移动 m 步定位，删除 $O(1)$ ，共 $n-1$ 次

2 题目二

2.1 题目表述

利用教材中的 Stack 类，为其设计外部函数（非成员函数）实现下面 delete_all 功能，必要时可以使用临时的 Stack 对象。编写主函数测试 delete_all 函数，栈元素设定为字符类型即可。

```
template <class T>
```

void delete_all(Stack<T> &s, const T &x)——删除栈 s 中所有等于 x 的数据项，保持其他数据项顺序不变。

输入：input.txt，其第一个字符为 x，其后按栈底到栈顶的顺序依次给出栈中字符，字符间用空格、回车或制表符间隔，如：

```
a
b a t a a e c
```

表示栈底栈顶内容为 b a t a a e c，要删除内容为 a

输出：删除后栈中字符内容，从栈顶到栈底的顺序即可，相邻元素间用空格间隔，最后一个元素之后不能有空格。最后输出一个回车。注意：应正确处理输入中空栈等情况。

2.2 代码 2 的测试用例

1. 输入：a

栈：b a t a a e c

输出：c e t b

2. 输入：a

栈：

输出：

3. 输入: z

栈: b a t a a e c

输出: c e a a t a b

4. 输入: a

栈: a a a

输出:

5. 输入: k

栈: a k b k c

输出: c b a

2.3 思路

1. 首先定义栈类 Stack 使用 vector 模拟栈的底层存储, 栈顶对应 vector 的尾部
2. 再实现 delete_all 函数

```
1 void delete_all(Stack& s, char x)
```

删除栈 s 中所有值为 x 的元素。

- 第一步: 将原栈 s 全部弹出, 跳过 x, 其余存入 temp1
通过不断 pop() 遍历原栈, 每个元素如果不等于 x, 就压入 temp1。
- 第二步: 把 temp1 倒入 temp2
把 temp1 所有元素移到 temp2, 相当于再次反转, 这时 temp2 中的元素顺序就是原始的从底到顶的顺序。
- 第三步: 把 temp2 移回原栈 s
再次转移, 恢复正确顺序并重建原栈。

3. 最后主函数 main() 执行流程

- 第一步: 打开输入文件, 读取目标字符和后续所有字符, 关闭文件
自动跳过空格、换行等空白符, 将所有读到的字符都放入 items 向量中。

```
1 vector<char> items;  
2 char c;  
3 while (infile >> c) {  
4     items.push_back(c);  
5 }
```

- 第二步: 构建栈
创建栈对象 s, 调用 loadFromBottom, 把 items 整体赋值给 data, 表示栈从底到顶的元素。
- 第三步: 调用 delete_all(Stack& s, char x) 函数删除所有 x
- 第四步: 按要求格式输出结果

2.4 测试用例截图

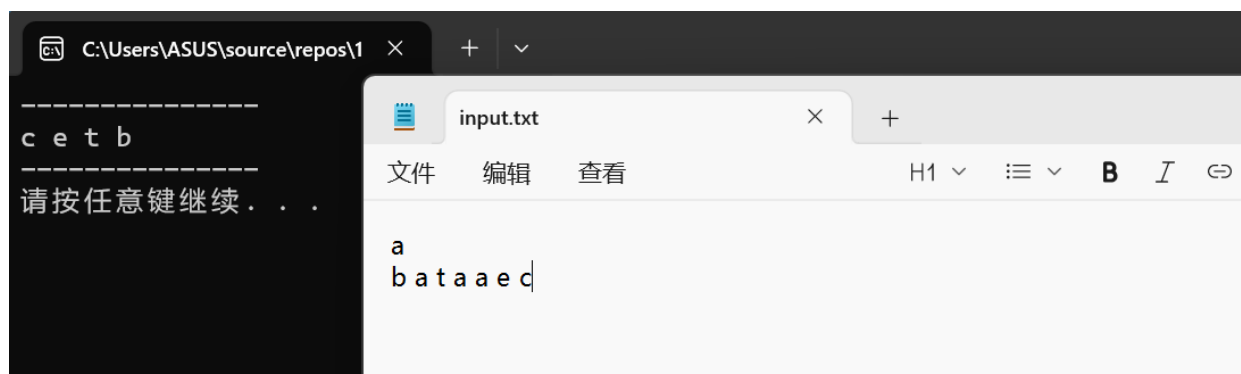


图 2.6: 题目二测试用例输出结果 1

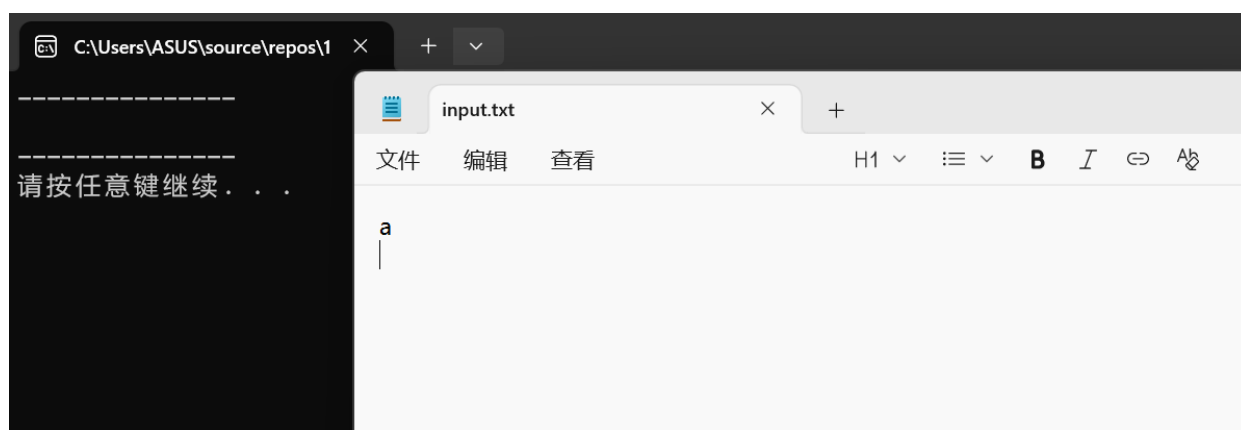


图 2.7: 题目二测试用例输出结果 2



图 2.8: 题目二测试用例输出结果 3

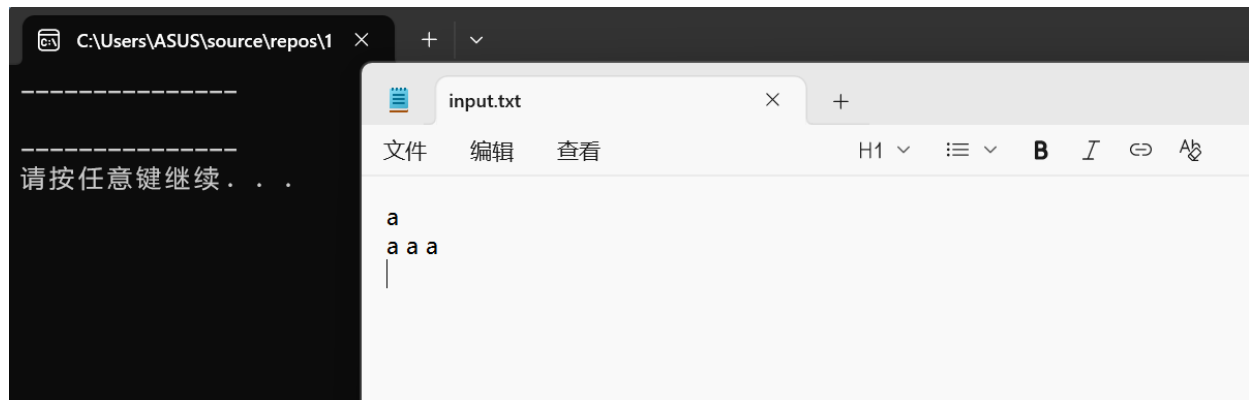


图 2.9: 题目二测试用例输出结果 4



图 2.10: 题目二测试用例输出结果 5

2.5 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 using namespace std;
5
6 class Stack {
7 private:
8     vector<char> data;
9 public:
10     Stack() {}
11     void push(char item) {
12         data.push_back(item);
13     }
14
15     void pop() {
16         if (!empty()) {
17             data.pop_back();
```

```
18     }
19 }
20
21 char& top() {
22     if (empty()) {
23         throw runtime_error("Stack is empty!");
24     }
25     return data.back();
26 }
27
28 const char& top() const {
29     if (empty()) {
30         throw runtime_error("Stack is empty!");
31     }
32     return data.back();
33 }
34
35 bool empty() const {
36     return data.empty();
37 }
38
39 int size() const {
40     return data.size();
41 }
42
43
44 void loadFromBottom(const vector<char>& items) {
45     data = items;
46 }
47
48
49 vector<char> getBottomToTop() const {
50     return data;
51 }
52 };
53
54
55 void delete_all(Stack& s, char x) {
56     Stack temp1;
57     Stack temp2;
58
59
```

```
60 while (!s.empty()) {
61     char c = s.top();
62     s.pop();
63     if (c != x) {
64         temp1.push(c);
65     }
66 }
67
68
69 while (!temp1.empty()) {
70     temp2.push(temp1.top());
71     temp1.pop();
72 }
73
74
75 while (!temp2.empty()) {
76     s.push(temp2.top());
77     temp2.pop();
78 }
79 }
80
81 int main() {
82     ifstream infile("input.txt");
83     if (!infile.is_open()) {
84         cout << "Cannot open input.txt!" << endl;
85         system("pause");
86         return 0;
87     }
88     char x;
89     infile >> x;
90     vector<char> items;
91     char c;
92     while (infile >> c) {
93         items.push_back(c);
94     }
95     infile.close();
96     Stack s;
97     s.loadFromBottom(items);
98     delete_all(s, x);
99     cout << "-----" << endl;
100     if (s.empty()) {
101         cout << endl;
```

```
102     }
103     else {
104         vector<char> output = s.getBottomToTop();
105         for (int i = 0; i < output.size(); ++i) {
106             cout << output[i];
107             if (i < output.size() - 1) {
108                 cout << " ";
109             }
110         }
111         cout << endl;
112     }
113     cout << "-----" << endl;
114     system("pause");
115     return 0;
116 }
```

2.6 复杂度分析

2.6.1 空间复杂度

空间复杂度： $O(n)$ ，其中 n 为需要入栈的字符数量。各个结构总空间理论上可达 $4n$ 左右，但由于常数倍不影响渐进复杂度，总的空间复杂度仍为 $O(n)$ 。

2.6.2 时间复杂度

时间复杂度： $O(n)$ ，其中 n 为需要入栈的字符数量。